

```
1  
2  
3 Calcolo Scientifico{  
4
```

```
5 [Python Parte3]  
6  
7  
8  
9  
10  
11  
12  
13  
14
```

```
    Array mono e bidimensionali: Assegnazione,  
    operazioni su array, funzioni su array
```

```
    }  
}
```

Overview:

L'algebra lineare è uno degli elementi costitutivi essenziali del calcolo scientifico e della matematica computazionale. Gli oggetti dell'algebra lineare sono vettori e matrici.

Il pacchetto (package) NumPy include tutti gli strumenti necessari per manipolare questi oggetti.

È possibile costruire vettori e matrici o alterare quelle esistenti ed eseguire le principali operazioni di algebra lineare (prodotto scalare, prodotto matrice-vettore e prodotto matrice-matrice).

Infine, sono disponibili varie funzioni per risolvere problemi di algebra lineare (calcolo di determinante, norme, soluzioni di sistemi lineari).

Per utilizzare il package NumPy è necessario importare il modulo:

```
import numpy as np
```

Array creation routines:

```
numpy.array(object, dtype=None)
```

```
a = np.array([1, 1, 1],  
             dtype=float) # vettore monodimensionale di float
```

```
b = np.array([[1, 1, 1]],  
             dtype=float) # vettore riga di float
```

```
c = np.array([[1],  
             [1],  
             [1]]) # vettore colonna di interi
```

```
a[0] = 0 # modifica del primo elemento del vettore a
```

- Gli **indici** delle componenti partono sempre da 0
- Si **accede** ai singoli elementi con l'indice tra **parentesi quadre**
- **np.array()** è una funzione! Non dimenticare le parentesi tonde!

Array creation routines:

```
numpy.array(object, dtype=None)
```

```
A = np.array([[1/2, 3/7, 9/14],  
              [2/7, 5/14, -2/7],  
              [-1/2, -3/14, 4/7]]) # matrice 3x3
```

```
A[0,1] = 0 # mod dell'elemento sulla prima riga, seconda colonna di A
```

```
B = np.array([[2, -3, 4],  
              [1, 6, -3],  
              [-3, 4, 8]]) # matrice 3x3
```

```
B[0][1] = 0 # mod dell'elemento sulla prima riga, seconda colonna di B
```

- La matrice è compresa tra parentesi quadre; ogni riga è, a sua volta, compresa tra parentesi quadre.
- Si **accede** ai singoli elementi specificando tra **parentesi quadre** l'indice di riga e di colonna.

Array creation routines:

```
numpy.linspace(start, stop, num)
```

Genera num punti equispaziati tra a e b. Se num=50 si può omettere.

Esempi:

```
a = np.linspace(2.0, 3.0, num=5)
```

```
OUTPUT: [2.   , 2.25, 2.5  , 2.75, 3.   ]
```

```
b = np.linspace(1, 10, num=10)
```

```
OUTPUT: [ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
```

Array creation routines:

`numpy.copy(a)`

Genera un `array copia` di `a`. Questo è equivalente a `np.array(a, copy=True)`

Esempio:

```
x = np.array([1, 2, 3]) # Crea un array x
```

```
y = x # Copia del riferimento
```

```
z = np.copy(x) # Copia dei valori
```

Osserva che, modificando x, y cambia ma z non cambia.

```
x[0] = 10
```

```
x[0] == y[0]
```

```
OUTPUT: True
```

```
x[0] == z[0]
```

```
OUTPUT: False
```

Array creation routines:

```
numpy.zeros(shape, dtype=float)
```

Genera un array di un dato shape, riempito di soli 0.

Esempio:

```
a = np.zeros(5)
```

```
OUTPUT: [ 0.,  0.,  0.,  0.,  0.]
```

```
b = np.zeros(5, dtype=int) # specifico il tipo di dato
```

```
OUTPUT: [0, 0, 0, 0, 0]
```

```
A = np.zeros((2, 1)) # 2 righe e 1 colonna, cioè vettore colonna
```

```
OUTPUT: [[ 0.],  
         [ 0.]]
```

Array creation routines:

`numpy.eye(n)`

Genera l'array identità: una matrice quadrata di `n` righe (e colonne) con 1 sulla diagonale principale .

Esempio:

```
np.eye(3)
```

```
OUTPUT: [[1.,  0.,  0.],  
         [0.,  1.,  0.],  
         [0.,  0.,  1.]]
```


Array creation routines:

```
numpy.full(shape, fill_value, dtype=None)
```

Genera un array di un determinato `shape` e `dtype`, riempito di `fill_value`.

Esempio:

```
A = np.full((2, 2), np.inf)
```

```
OUTPUT: [[inf, inf],  
         [inf, inf]]
```

```
B = np.full((2, 2), 10)
```

```
OUTPUT: [[10, 10],  
         [10, 10]]
```

```
numpy.full((n,m), 1)  
equivale a  
numpy.ones((n,m))
```

Array attributes:

`numpy.ndarray.shape`

Tupla delle dimensioni dell'array. Lo shape di una matrice $m \times n$ è la tuple (m,n) . Per accedere ad un elemento della tuple, usiamo le parentesi quadre e l'indice

Esempio:

```
B = np.array([[2, -3, 4],  
              [1, 6, -3],  
              [-3, 4, 8]])
```

```
x, y = B.shape # in questo caso, uso l'attributo shape di B  
OUTPUT: (3, 3)
```

```
np.shape(B)[0] # qui utilizziamo la funzione shape di numpy  
OUTPUT: 3
```

Array attributes:

```
numpy.size(obj, axis=None)
```

Restituisce il numero di elementi in obj (è una funzione!).
Specificando axis, otteniamo il numero di elementi sull'asse.

Esempio:

```
B = np.array([[2, -3, 4],  
              [1, 6, -3],  
              [-3, 4, 8]])
```

```
np.size(B)
```

OUTPUT: 9

```
np.size(B, 0)
```

OUTPUT: 3

Array attributes:

`numpy.ndarray.T`

Array *trasposto*.

Esempio:

```
x = np.array([[1.,2.],  
              [3.,4.]])
```

```
y = np.copy(x.T) # x non viene modificato
```

```
OUTPUT: [[1. 3.]  
         [2. 4.]
```

Che succede se uso `y = x.T` e modifico `y`?

Sottomatrici:

Sia A un array bidimensionale. Siano x e y array.

A[np.ix_(x, y)] è la *sottomatrice* che si ottiene utilizzando gli elementi di x come indici di riga e quelli di y come indici di colonna.

Esempio:

```
A = np.array([[1, 2, 3, 4],  
              [5, 6, 7, 8],  
              [9, 10, 11, 12],  
              [13, 14, 15, 16]])
```

```
x = np.array([0, 2])
```

```
y = np.array([0, 2])
```

```
B = np.array( A[np.ix_(x, y)] )
```

OUTPUT:

```
[[ 1  3]  
 [ 9 11]]
```

Sottomatrici:

Sia A un array bidimensionale e i, j interi. È possibile ottenere sottomatrici usando lo `slice`:

$M[i,:]$ è l'array contenente la riga i di M .

$M[:,j]$ è l'array (riga) contenente la colonna j di M .

$M[0:2,:]$ è l'array bidimensionale contenente le righe 0 e 1 di M .

$M[2:4,1:4]$ è uno `slice` di righe e colonne.

NB: il secondo membro dello slice viene "escluso"

NB: gli indici partono da 0

Sottomatrici:

```
A = np.array([[1, 2, 3, 4],  
              [5, 6, 7, 8],  
              [9, 10, 11, 12],  
              [13, 14, 15, 16]])
```

```
b = np.array(A[1, :])  
print(b)
```

```
c = np.array(A[:, 1])  
print(c)
```

```
d = np.array(A[0:2, :])  
print(d)
```

```
e = np.array(A[0:2, 1:3])  
print(e)
```

OUTPUT:

```
[5 6 7 8]
```

```
[ 2  6 10 14]
```

```
[[1 2 3 4]  
 [5 6 7 8]]
```

```
[[2 3]  
 [6 7]]
```

Sottomatrici:

In figura sono riportati altri esempi di slice su array bidimensionali.

L'uso combinato di `colon` (`:`) e `slice` (`:`) permette di accedere facilmente a sottomatrici, minimizzando l'uso dei loop e migliorando la leggibilità del codice.

NB: `[-1,:]` è l'ultima riga della matrice.

`M[:2,1:-1]`

0	1	2	3
4	5	6	7
8	9	10	11

`M[1:2,:]`

0	1	2	3
4	5	6	7
8	9	10	11

`M[1,:]`

0	1	2	3
4	5	6	7
8	9	10	11

`M[1:2,1:2]`

0	1	2	3
4	5	6	7
8	9	10	11

`M[1,1]`

0	1	2	3
4	5	6	7
8	9	10	11

Esercizi {

Qual è l'output di questi comandi?

```
A = np.array([[1, 2, 3, 4],  
              [5, 6, 7, 8],  
              [9, 10, 11, 12],  
              [13, 14, 15, 16]])
```

```
print(A[0, 0])  
print(A[4][1])  
print(A[4, 1])  
print(A[1:2, :])  
print(A[:, 2])  
print(A[:, 0:2])  
print(A[np.ix_([0, 2], [1, 3])])  
print(A[:, 0:2])  
print(A[:, 0:2])
```

}

Operazioni algebriche:

Le principali operazioni algebriche tra matrici o vettori, o tra matrici/vettori e scalari sono eseguite automaticamente. Le operazioni tra matrici sono consentite solo se le dimensioni sono compatibili!

Le operazioni si dicono essere element-wise, ovvero elemento per elemento.

Ad esempio, se A e B sono matrici di uguali dimensioni,

$C = A+B$ è calcolato come $C[i][j] = A[i][j] + B[i][j]$

$C = A*B$ è calcolato come $C[i][j] = A[i][j] * B[i][j]$

$C = A**3$ è calcolato come $C[i][j] = A[i][j]^3$

Operazioni su matrici:

Le principali operazioni proprie di matrici e vettori sono eseguite tramite funzioni.

Ad esempio, se A e B sono matrici/vettori compatibili:

$A @ B$ è il prodotto righe per colonne

`np.linalg.inv(A)` è l'inversa di A

`np.linalg.det(A)` è il determinante di A

`A.transpose()` o `A.T` è la trasposta di A

`np.linalg.matrix_rank(A)` è il rango di A

`D, V = np.linalg.eig(a)` restituisce autovelori e autovettori di A

1 Confronto tra matrici:

2 Fra due matrici di uguali dimensioni si possono utilizzare
3 gli operatori relazionali.

4
5 < minore
6 <= minore o uguale
7 > maggiore
8 >= maggiore o uguale
9 == uguale
10 != diverso

11 Il confronto avviene elemento per elemento ed il risultato è
12 una matrice booleana, ovvero una matrice contenente solo True
13 o False.

Array manipulation routines:

Sia M un array bidimensionale (matrice).

$x = \text{np.diag}(M, k)$ è l'array contenente gli elementi sulla k -ma diagonale.

$A = \text{np.tril}(M, k)$ estrae il triangolo inferiore di M partendo dalla k -ma diagonale.

$B = \text{np.triu}(M, k)$ estrae il triangolo superiore di M partendo dalla k -ma diagonale.

Se k è omissso, si intende $k=0$ ovvero la diagonale principale.

1 Altre funzioni:

2
3 Sia M un array bidimensionale (matrice).

4
5 `np.sort(M)` ordina gli elementi in ordine crescente

6 `np.mean(M)` calcola la media degli elementi

7 `np.argmax(M)` restituisce l'indice dell'elemento massimo

8 `np.amax(M)` restituisce il valore massimo

9 Per ogni funzione, è possibile specificare l'asse su cui calcolarle.

10 `np.linalg.norm(M, ord)` calcola la norma di M secondo l'ordine
11 specificato (in generale, ponete `ord = np.inf`)

12 `np.linalg.cond(M)` calcola l'indice di condizionamento di M

13 `np.linalg.solve(A, b)` restituisce la soluzione del sistema lineare $Ax=b$
14

Resources {



Link:

- * NumPy Reference:
<https://numpy.org/doc/stable/reference/index.html#reference>
- * Pagina elearning del corso



Riferimenti testuali:

- * Fuhrer, Verdier, Solem, Scientific Computing with Python,
cap 4

}

Credits {

Il template della presentazione è stato
creato da Slidesgo <https://slidesgo.com/>

I contenuti sono stati sviluppati durante il corso
di Calcolo Scientifico per Laurea Triennale in
Informatica, docente prof. Dajana Conte, da Tomeo
Fiorentino: <https://github.com/FioreTom>



}